



ติวเตอร์แมทซ์

จัดทำโดย

นายกฤตยชญ์ แก้วกำมา รหัสนักศึกษา 6710535011

นายวุฒิสักดิ์ บุญกัน รหัสนักศึกษา 6710535029

นายภูวรัตน์ นาคจันทิก รหัสนักศึกษา 6710615201

นายธนกฤต โพธิมาศ รหัสนักศึกษา 6710625010

เสนอ

รศ.ดร.วีรชัย อโณทัยไพบูลย์

รายงานนี้เป็นส่วนหนึ่งของวิชา วพ.230 ระบบฐานข้อมูล
หลักสูตรวิศวกรรมคอมพิวเตอร์ คณะวิศวกรรมศาสตร์ มหาวิทยาลัยธรรมศาสตร์
ภาคเรียนที่ 2 ปีการศึกษา 2568

คำนำ

รายงานฉบับนี้จัดทำขึ้นเป็นส่วนหนึ่งของรายวิชา วพ.230 ระบบฐานข้อมูล หลักสูตร วิศวกรรมคอมพิวเตอร์ คณะวิศวกรรมศาสตร์ มหาวิทยาลัยธรรมศาสตร์ ประจำปีการศึกษา 2568 โดยมีวัตถุประสงค์เพื่อศึกษา วิเคราะห์ ออกแบบ และพัฒนาระบบฐานข้อมูลรวมกับการประยุกต์ใช้ความรู้ทางด้านการพัฒนาซอฟต์แวร์ ผ่านโครงการที่มีชื่อว่า ดิวเตอร์แมทซ์

โครงการนี้เกิดขึ้นจากการเล็งเห็นปัญหาในการหาตัวเตอร์และการหางานสอนพิเศษในปัจจุบัน ซึ่งยังคงมีข้อจำกัดในเรื่องของการเข้าถึงข้อมูล การติดต่อสื่อสารระหว่างนักเรียนและตัวเตอร์ รวมถึงการพึ่งพาตัวกลางที่มีค่าธรรมเนียมสูงและอาจทำให้เกิดความไม่สะดวกในการดำเนินการ ผู้จัดทำจึงมีแนวคิดในการพัฒนาระบบแพลตฟอร์มออนไลน์ที่ทำหน้าที่เป็นตัวกลางในการจับคู่ระหว่างนักเรียนและตัวเตอร์โดยตรง เพื่อช่วยเพิ่มความสะดวก ลดต้นทุน และสร้างความโปร่งใสในการใช้งาน

ภายในรายงานเล่มนี้ประกอบไปด้วยเนื้อหาเกี่ยวกับการศึกษาปัญหาและความต้องการของระบบ การวิเคราะห์กระบวนการทำงาน การออกแบบฐานข้อมูลตามหลักการทำ Normalization การออกแบบความสัมพันธ์ของข้อมูล การพัฒนาโครงสร้างระบบทั้งส่วน Frontend และ Backend ตลอดจนการทดสอบระบบและการวิเคราะห์ผลลัพธ์ที่ได้จากการดำเนินโครงการ เพื่อให้ระบบสามารถทำงานได้อย่างมีประสิทธิภาพ ถูกต้อง และสามารถนำไปพัฒนาต่อยอดได้ในอนาคต

ผู้จัดทำหวังเป็นอย่างยิ่งว่ารายงานฉบับนี้จะเป็นประโยชน์ต่อผู้อ่าน ไม่ว่าจะเป็นนักเรียน นักศึกษา หรือผู้ที่สนใจศึกษาเกี่ยวกับการออกแบบระบบฐานข้อมูลและการพัฒนาแพลตฟอร์มออนไลน์ หากมีข้อผิดพลาดประการใด ผู้จัดทำขอน้อมรับคำแนะนำและข้อเสนอแนะไว้เพื่อปรับปรุงและพัฒนาต่อไปในอนาคต

คณะผู้จัดทำ ดิวเตอร์แมทซ์

สารบัญ

	หน้า
คำนำ	ก
สารบัญ	๗
บทที่ 1 บทนำ	1
1.1 ความเป็นมาและความสำคัญของปัญหา	1
1.2 วัตถุประสงค์ของโครงการ	1
1.3 ขอบเขตของโครงการ	1
1.4 สมมติฐาน	2
1.5 นิยามศัพท์เฉพาะ	2
1.6 ประโยชน์ที่คาดว่าจะได้รับ	2
บทที่ 2 ทฤษฎีและเทคโนโลยีที่เกี่ยวข้อง	3
2.1 แนวคิดแพลตฟอร์มตัวกลาง	3
2.2 ทฤษฎีการออกแบบฐานข้อมูลและการจัดระเบียบข้อมูล	3
2.3 ระบบยืนยันตัวตนแบบไร้สถานะ	3
2.4 ความสมบูรณ์ของข้อมูลและระบบตรวจสอบ	4
2.5 เทคโนโลยีที่ใช้ในการพัฒนา	4
บทที่ 3 การออกแบบระบบ	5
3.1 ภาพรวมของระบบ	5
3.2 สถาปัตยกรรมระบบ	5
3.3 ผู้มีส่วนได้ส่วนเสีย	5
3.4 ความต้องการของระบบ	7
3.5 การออกแบบฐานข้อมูล	8
3.5.1 Database Normalization	10
3.5.2 Database Optimization	11
3.5.3 รายละเอียดโครงสร้าง Entity	12
3.5.4 สรุป Cardinality	14
3.6 System Integrity	16
3.6.1 ระบบป้องกันการรั่วซึม	16
3.6.2 ระบบป้องกันการโจมตีและจำกัดโควตา	16
3.6.3 ระบบรักษาสถานะและความปลอดภัย	16

	หน้า
บทที่ 4 การพัฒนาและผลการทำงานของระบบ	17
4.1 ตัวอย่าง Use Case หลักของระบบ	17
4.2 System Flow.....	17
4.3 การทำงานของระบบหน้าบ้าน.....	20
4.4 การจัดการตรรกะระบบหลังบ้าน	20
4.5 การจัดการข้อผิดพลาด.....	20
บทที่ 5 สรุปผลและข้อเสนอแนะ	21
5.1 สรุปผลการดำเนินงาน.....	21
5.2 ข้อจำกัดของระบบ	21
5.3 ข้อเสนอแนะเพื่อการพัฒนาต่อ.....	22
บรรณานุกรม	23
ภาคผนวก.....	24
ภาคผนวก ก: E/R Diagram ก่อนทำการ Normalization	24
ภาคผนวก ข: E/R Diagram หลังทำการ Normalization.....	25
ภาคผนวก ค: รายการอ้างอิงโครงสร้างตาราง Database Schema	26
ภาคผนวก ง: รายการอ้างอิง SQL Queries	28

บทที่ 1 บทนำ

1.1 ความเป็นมาและความสำคัญของปัญหา

ในปัจจุบันการหาติวเตอร์หรือการหางานสอนพิเศษยังคงมีข้อจำกัดหลายประการ โดยเฉพาะเรื่องการเข้าถึงข้อมูลและช่องทางในการติดต่อระหว่างนักเรียนและติวเตอร์ ซึ่งส่วนใหญ่มักต้องอาศัยตัวกลางหรือนายหน้าในการประสานงาน ทำให้เกิดค่าใช้จ่ายเพิ่มเติม และบางครั้งข้อมูลที่ได้รับก็ไม่ใช่ประโยชน์หรือไม่ตรงกับความต้องการจริง จากปัญหาดังกล่าว ผู้จัดทำจึงมีแนวคิดที่จะพัฒนาระบบแพลตฟอร์มออนไลน์ที่สามารถเชื่อมต่อระหว่างนักเรียนและติวเตอร์ได้โดยตรง โดยไม่จำเป็นต้องผ่านตัวกลาง เพื่อช่วยลดค่าใช้จ่าย เพิ่มโอกาสในการเข้าถึงงานสอน และทำให้ทั้งสองฝ่ายสามารถติดต่อกันได้อย่างสะดวกมากยิ่งขึ้น

ถ้าจะเปรียบเทียบในเรื่องของความต้องการทางการตลาดของติวเตอร์ ในปัจจุบัน ตลาดของการเรียนพิเศษมีลักษณะเป็น Demand และ Supply ที่ใกล้เคียงกัน กล่าวคือ มีนักเรียนจำนวนมากที่ต้องการหาติวเตอร์ (Demand) และในขณะเดียวกันก็มีติวเตอร์จำนวนมากที่ต้องการหางานสอน (Supply) แต่ปัญหาที่เกิดขึ้นคือทั้งสองฝ่ายไม่สามารถเข้าถึงกันได้โดยตรงอย่างมีประสิทธิภาพ

ช่องว่างนี้ทำให้เกิด "ตัวกลาง" หรือ "นายหน้า" เข้ามามีบทบาทในการจับคู่ระหว่างนักเรียนและติวเตอร์ ซึ่งแม้จะช่วยให้เกิดการเชื่อมต่อ แต่ก็มักมาพร้อมกับค่าธรรมเนียมที่ค่อนข้างสูง และบางครั้งข้อมูลที่ส่งต่อก็คงอาจไม่ครบถ้วนหรือไม่ตรงกับความต้องการของผู้ใช้งานจริง

1.2 วัตถุประสงค์ของโครงการ

สร้างแพลตฟอร์มที่เป็นสื่อกลาง (Matchmaking) พร้อมระบบยืนยันตัวตนและการจัดการตารางเวลา โดยมีเป้าหมายหลักในการลดปัญหาการเข้าถึงตลาด ลดการพึ่งพานายหน้าที่มีค่าธรรมเนียมสูง รวมทั้งมีระบบรีวิวเพื่อเพิ่มความน่าเชื่อถือของติวเตอร์

1.3 ขอบเขตของโครงการ (Scope)

ระบบครอบคลุมการทำงานของผู้ใช้งาน 3 บทบาทหลัก ได้แก่

1. ผู้เรียน (Student)

- สร้างและจัดการโพสต์ประกาศหาติวเตอร์ โดยระบุรายละเอียดวิชา ระดับชั้น วิชา เวลา งบประมาณ
- ค้นหาและกรองรายชื่อติวเตอร์ ดูโปรไฟล์และรีวิว
- เลือกติวเตอร์ที่มาสัมภาษณ์ จองเวลาเรียน ชำระเงินผ่านระบบ
- เขียนรีวิวและให้คะแนนหลังเรียนจบ

2. ผู้ใช้งาน (Tutor)

- สร้างโปรไฟล์ ระบุประวัติการศึกษา ประสบการณ์ ราคา อับโหลดเอกสาร
- ค้นหาโพสต์และสมัครงานสอน
- ตั้งค่าตารางเวลา
- รับเงินค่าสอนหลังหักค่าธรรมเนียมแพลตฟอร์ม

3. ผู้ดูแลระบบ (Admin)

- ตรวจสอบและอนุมัติเอกสารผู้ใช้งาน
- จัดการผู้ใช้งานที่ผิดปกติ ซ่อนโพสต์หรือรีวิวที่ไม่เหมาะสม
- ดูสถิติ Dashboard ของระบบ

1.4 สมมติฐาน

ระบบฐานข้อมูลที่ออกแบบตามหลัก Normalization ถึงระดับ 4NF จะสามารถลดความซ้ำซ้อนของข้อมูลและป้องกัน Anomaly ได้อย่างมีประสิทธิภาพ และการใช้ระบบ Escrow ในการจัดการการเงินจะช่วยสร้างความเชื่อมั่นให้กับผู้ใช้งานทั้งสองฝ่าย

1.5 นิยามศัพท์เฉพาะ

1. **Matchmaking Platform** — แพลตฟอร์มที่ทำหน้าที่เป็นตัวกลางจับคู่ระหว่างนักเรียนและผู้สอน
2. **Escrow** — ระบบที่แพลตฟอร์มถือเงินไว้ก่อน แล้วโอนให้ผู้สอนหลังจากการสอนเสร็จสิ้นและได้รับการยืนยัน
3. **Verification** — กระบวนการตรวจสอบและยืนยันตัวตนของผู้สอนโดย Admin
4. **Platform Fee** — ค่าธรรมเนียมที่ระบบหักจากรายได้ผู้สอนในอัตรา 10%
5. **Soft Delete** — การซ่อนหรือระงับข้อมูลแทนการลบออกจากฐานข้อมูลจริง เพื่อรักษา Referential Integrity
6. **JWT (JSON Web Token)** — มาตรฐานการยืนยันตัวตนแบบไร้สถานะที่ใช้ในระบบ

1.6 ประโยชน์ที่คาดว่าจะได้รับ

1. นักเรียนและผู้สอนสามารถติดต่อกันได้โดยตรง ลดต้นทุนและเพิ่มความโปร่งใส
2. ระบบรีวิวช่วยสร้างความน่าเชื่อถือและมาตรฐานคุณภาพของผู้สอน
3. ระบบการเงินแบบ Escrow ช่วยปกป้องทั้งนักเรียนและผู้สอน
4. ฐานข้อมูลที่ออกแบบอย่างเป็นระบบรองรับการขยายตัวของแพลตฟอร์มในอนาคต

บทที่ 2 ทฤษฎีและเทคโนโลยีที่เกี่ยวข้อง

การพัฒนาแพลตฟอร์ม Tutor Match จำเป็นต้องอาศัยความรู้ทางด้านวิศวกรรมซอฟต์แวร์ และการจัดการฐานข้อมูลขั้นสูง เพื่อให้บรรลุเป้าหมายในการเป็นตัวกลางที่น่าเชื่อถือระหว่างนักเรียน และติวเตอร์ โดยมีรายละเอียดทฤษฎีและเทคโนโลยีที่เกี่ยวข้องดังนี้

2.1 แนวคิดแพลตฟอร์มตัวกลาง (Matchmaking Platform)

ระบบถูกออกแบบมาเพื่อลดปัญหาความเหลื่อมล้ำในการเข้าถึงตลาดการศึกษา และลดภาระ ค่าธรรมเนียมจากนายหน้า โดยเน้นการสร้างความต้องการ (Demand) จากฝั่งนักเรียนผ่านการโพสต์ ประกาศ และการตอบสนอง (Supply) จากฝั่งติวเตอร์ผ่านการสมัครสอน

2.2 ทฤษฎีการออกแบบฐานข้อมูลและการจัดระเบียบข้อมูล (Database Normalization)

เพื่อให้ฐานข้อมูลปราศจากความซ้ำซ้อนและป้องกันความผิดพลาดในการจัดการข้อมูล (Anomaly) ระบบจึงใช้หลักการ Normalization ดังนี้

1. **Third Normal Form (3NF)** นำมาใช้ในตาราง Review และ Payment โดยการตัด ข้อมูลที่สามารถสืบค้นได้จาก app_id ออก เพื่อลด Transitive Dependency
2. **Boyce-Codd Normal Form (BCNF)** ประยุกต์ใช้ในตาราง Tutor_Subject และ Tutor_Certificate โดยการใช้ Composite Primary Key เพื่อป้องกันการบันทึกข้อมูล ซ้ำซ้อนของติวเตอร์คนเดิม
3. **Fourth Normal Form (4NF)** ใช้ในการจัดการข้อมูล Multivalued ในตาราง Tutor_Experience โดยแยกรายละเอียดประสบการณ์การสอนออกมาเป็นตารางย่อย เพื่อให้การสืบค้นและการเก็บข้อมูลมีประสิทธิภาพสูงสุด

2.3 ระบบยืนยันตัวตนแบบไร้สถานะ (Stateless Authentication)

ระบบเลือกใช้เทคโนโลยี JSON Web Token (JWT) แทนการใช้ Session แบบดั้งเดิม เพื่อ ลดภาระการทำงานของฐานข้อมูล (Zero Database Hit for Authorization) โดย Token จะเก็บ ข้อมูลเบื้องต้น เช่น user_id และ role ไว้ที่ฝั่งผู้ใช้งาน ทำให้เซิร์ฟเวอร์สามารถตรวจสอบสิทธิ์ได้ทันที ผ่าน Secret Key

2.4 ความสมบูรณ์ของข้อมูลและระบบตรวจสอบ (Data Integrity & Audit Trail)

1. **Database Constraints** มีการบังคับใช้กฎในระดับโครงสร้าง เช่น CHECK เพื่อควบคุมงบประมาณและราคาให้ถูกต้องตามตรรกะธุรกิจ และ UNIQUE เพื่อป้องกันการสมัครงานซ้ำในโพสต์เดิม
2. **Enterprise Auditability** การออกแบบตาราง User_Action_Log แยกออกมาเพื่อบันทึกประวัติการดำเนินการที่สำคัญของ Admin เช่น การแบนผู้ใช้ หรือการอนุมัติโปรไฟล์ติวเตอร์ เพื่อให้สามารถตรวจสอบย้อนหลังได้ 100%
3. **Soft Delete** การใช้สถานะบัญชี (account_status) แทนการลบแถวข้อมูลจริง เพื่อรักษาความสัมพันธ์ของการอ้างอิง (Referential Integrity) และไม่ให้สถิติทางการเงินสูญหาย

2.5 เทคโนโลยีที่ใช้ในการพัฒนา (Tech Stack)

1. เทคโนโลยีฝั่งส่วนติดต่อผู้ใช้งาน (Frontend Development)

พัฒนา UI ด้วย HTML (HyperText Markup Language) และ CSS (Cascading Style Sheets) โดยใช้ JavaScript ควบคุมตรรกะฝั่ง Client และเชื่อมต่อ API แบบอซิงโครนัส (Asynchronous) เพื่อให้ระบบทำงานรวดเร็วและสลับไหลดโดยไม่ต้องโหลดหน้าเว็บใหม่ (Reload) ทุกครั้งที่มีการเปลี่ยนแปลงข้อมูลเทคโนโลยีฝั่งระบบหลังบ้าน (Backend)

2. เทคโนโลยีฝั่งระบบหลังบ้าน (Backend Development)

พัฒนาด้วยภาษา Python โดยประยุกต์ใช้ Flask Framework ซึ่งเป็นสถาปัตยกรรมแบบไมโครเฟรมเวิร์ก (Microframework) เพื่อช่วยในการสร้างและให้บริการ API (RESTful API) ที่มีความยืดหยุ่นสูง รวดเร็ว และสามารถรองรับการเขียนตรรกะทางธุรกิจ (Business Logic) ที่ซับซ้อนได้อย่างมีประสิทธิภาพ

3. ระบบจัดการฐานข้อมูล (Database Management System)

เลือกใช้ระบบจัดการฐานข้อมูลเชิงสัมพันธ์ (Relational Database) MySQL เพื่อจัดเก็บข้อมูลของระบบ โดยออกแบบโครงสร้างสถาปัตยกรรมข้อมูลให้รองรับกระบวนการลดความซ้ำซ้อน (Database Normalization) ไปจนถึงระดับ BCNF และ 4NF ซึ่งช่วยรักษาความถูกต้องสมบูรณ์ของข้อมูล (Data Integrity) และเพิ่มประสิทธิภาพในการสืบค้นข้อมูลขั้นสูง

4. มาตรฐานการรักษาความปลอดภัย (Security and Encryption)

ระบบให้ความสำคัญกับความปลอดภัยของข้อมูลส่วนบุคคลเป็นหลัก โดยมีการประยุกต์ใช้อัลกอริทึม Bcrypt ในการทำกระบวนการแฮชซิง (Password Hashing) เพื่อเข้ารหัสผ่านของผู้ใช้งานก่อนบันทึกลงสู่ฐานข้อมูล ซึ่งช่วยป้องกันการถูกโจมตีแบบส่มรหัสผ่าน (Brute-force Attack) และยกระดับความปลอดภัยของแพลตฟอร์ม

บทที่ 3 การออกแบบระบบ (System Design & Methodology)

3.1 ภาพรวมของระบบ (System Overview)

ระบบนี้ถูกออกแบบมาเพื่อเป็นแพลตฟอร์มกลางสำหรับเชื่อมต่อระหว่างนักเรียนและติวเตอร์โดยตรง ช่วยให้นักเรียนสามารถโพสต์หาติวเตอร์ตามความต้องการ และติวเตอร์สามารถค้นหาพร้อมสมัครงานสอนได้โดยตรง โดยมีเป้าหมายหลักในการลดปัญหาการเข้าถึงตลาด ลดการพึ่งพานายหน้าที่มีค่าธรรมเนียมสูง รวมทั้งมีระบบรีวิวเพื่อเพิ่มความน่าเชื่อถือของติวเตอร์

3.2 สถาปัตยกรรมระบบ (System Architecture)

ระบบใช้สถาปัตยกรรมแบบแบ่งชั้น (Layered Architecture) ออกเป็น 3 ชั้นหลัก

1. **ชั้น Routes** รับ HTTP Request จาก Frontend ตรวจสอบ Input และส่งต่อไปยัง Service Layer พร้อม JSON Response กลับไปยัง Client
2. **ชั้น Services** เป็นที่รวม Business Logic ทั้งหมด แต่ละ service แยกตามบทบาทของผู้ใช้ (auth_service, student_service, tutor_service, payment_service, review_service, admin_service)
3. **ชั้น Database** เชื่อมต่อกับ MySQL ผ่าน Connection Pooling เพื่อ reuse connection ต่อ request โดยใช้ PyMySQL

3.3 ผู้มีส่วนได้ส่วนเสีย (Stakeholders)

1. นักเรียน (Student)

- บทบาท: ผู้ใช้งานฝั่งผู้เรียนที่ต้องการหาความรู้เพิ่มเติม และเป็นแหล่งที่มาของรายได้ในระบบ
- การกระทำในระบบ:
 - **สร้างความต้องการ** สามารถตั้งโพสต์ประกาศหาติวเตอร์ โดยระบุรายละเอียดวิชา ระดับชั้น วันเวลาที่สะดวก และงบประมาณที่ต้องการได้
 - **ค้นหาและตัดสินใจ** สามารถค้นหาและกรองรายชื่อติวเตอร์จากระบบ ดูโปรไฟล์ ประวัติการสอน และอ่านรีวิวจากนักเรียนคนอื่นๆ เพื่อประกอบการตัดสินใจ
 - **ดำเนินการและชำระเงิน** ทำการเลือกติวเตอร์ที่มาสมัคร จองเวลาเรียนตามตารางที่ติวเตอร์ระบุไว้ว่าง และทำการชำระเงินค่าเรียนผ่านระบบ
 - **การประเมินผล** เป็นผู้ให้คะแนน (Rating) และเขียนรีวิว (Review) ให้กับติวเตอร์หลังจากเรียนจบ เพื่อสร้างประวัติความน่าเชื่อถือในระบบ

2. ดิวเตอร์ (Tutor)

- บทบาท: ผู้ให้บริการสอนหนังสือที่ต้องการหาผู้เรียนและสร้างรายได้ผ่านแพลตฟอร์ม
- การกระทำในระบบ:
 - **สร้างความน่าเชื่อถือ** สร้างโปรไฟล์เพื่อนำเสนอตัวเอง ระบุประวัติการศึกษา ประสบการณ์ เทรนาคาต่อชั่วโมง และอัปโหลดเอกสารสำคัญเพื่อยืนยันตัวตนจาก Admin
 - **หางานสอน** ค้นหาโพสต์ประกาศหาติวเตอร์ของนักเรียนที่ตรงกับความถนัดของตนเอง และกดส่งคำขอสมัครสอน (Apply)
 - **จัดการเวลา** ตั้งค่าและอัปเดตตารางเวลา (Schedule) ของตนเองว่าวันและเวลาใดบ้างที่สะดวกรับงานสอน
 - **รับรายได้** ตกลงรับงานสอน และรับเงินค่าสอนหลังจากที่ระบบทำการหักค่าธรรมเนียมการใช้แพลตฟอร์ม (Platform Fee) เรียบร้อยแล้ว

3. ผู้ดูแลระบบ (Admin)

- บทบาท: ผู้ควบคุมดูแลความเรียบร้อย บริหารจัดการข้อมูลหลังบ้าน และวิเคราะห์การเติบโตของแพลตฟอร์ม
- การกระทำในระบบ:
 - **การตรวจสอบและอนุมัติ** ตรวจสอบความถูกต้องของเอกสารที่ติวเตอร์อัปโหลดเข้ามา และปรับสถานะโปรไฟล์ให้เป็น "ยืนยันตัวตนแล้ว (Verified)"
 - **การจัดการผู้ใช้และเนื้อหา (Moderation)** ดูแลความเรียบร้อยของระบบ สามารถลบโพสต์ที่ไม่เหมาะสม ลบรีวิวที่เป็นสแปม หรือระงับบัญชีผู้ใช้งาน
 - **วิเคราะห์ข้อมูล (Analytics)** ดูหน้าแดชบอร์ดสรุปสถิติสำคัญ เช่น รายได้รวมของระบบ สัดส่วนผู้ใช้งาน จำนวนติวเตอร์ที่รอการอนุมัติ

3.4 ความต้องการของระบบ (Functional Requirements)

1. การจัดการบัญชีผู้ใช้ (User Account Management)

- การสมัครสมาชิกและยืนยันตัวตน: ผู้ใช้สามารถสมัครสมาชิกใหม่และเข้าสู่ระบบด้วยอีเมลและรหัสผ่านได้
- การแบ่งสิทธิ์ผู้ใช้งาน (Role-Based Access): ระบบต้องสามารถจำกัดสิทธิ์การเข้าถึงเมนูต่างๆ ตามบทบาทของผู้ใช้ ได้แก่ นักเรียน (Student), ทิวเตอร์ (Tutor), และผู้ดูแลระบบ (Admin)
- การรักษาความปลอดภัย: ระบบต้องทำการเข้ารหัสผ่าน (Password Hashing) ก่อนบันทึกลงฐานข้อมูล

2. การจัดการโปรไฟล์ผู้ใช้งาน (User Profile Management)

- การสร้างโปรไฟล์นักเรียน (Student Profile): นักเรียนสามารถระบุข้อมูลระดับชั้นเรียน และชื่อโรงเรียนหรือมหาวิทยาลัย
- การสร้างโปรไฟล์ทิวเตอร์ (Tutor Profile): ทิวเตอร์สามารถใส่ข้อมูลความแนะนำตัว ประสบการณ์การสอน กำหนดเรทราคาค่าสอนต่อชั่วโมง รวมถึงเพิ่มรายชื่อวิชาที่รับสอน และอัปโหลดเอกสารใบรับรองต่างๆ

3. การจัดการโพสต์หาทิวเตอร์ (Student Post Management)

- การสร้างและจัดการโพสต์: นักเรียนสามารถสร้างโพสต์โดยระบุวิชาที่ต้องการเรียน รายละเอียดเพิ่มเติม และงบประมาณต่อชั่วโมง
- กระดานงานสอน (Job Board): ระบบต้องมีหน้าแสดงรายการโพสต์หางานสอนทั้งหมดที่ยังเปิดรับสมัครอยู่

4. การจัดการการสมัครงานสอน (Job Application Management)

- การส่งคำขอสมัครงาน: ทิวเตอร์สามารถกดปุ่มสมัครงานในโพสต์ที่สนใจได้
- การจัดการสถานะการสมัคร: ระบบต้องแสดงรายชื่อและโปรไฟล์ของทิวเตอร์ที่มาสมัครทั้งหมดให้นักเรียนเห็น และให้นักเรียนสามารถกดเลือก "ยอมรับ" หรือ "ปฏิเสธ"

5. ระบบจัดการตารางเวลาเรียน (Tutor Availability & Scheduling)

- การตั้งค่าเวลาว่าง: ตัวเตอรสามารถกำหนดวันในสัปดาห์ (จันทร์-อาทิตย์) และช่วงเวลา (เวลาเริ่ม-เวลาสิ้นสุด) ที่ตนเองสะดวกรับงานสอน
- การจองเวลาและการป้องกันการจองซ้ำซ้อน (Schedule Booking & Conflict Prevention)

6. ระบบการจัดการชำระเงิน (Payment Management)

- การชำระค่าเรียน: นักเรียนสามารถชำระเงินค่าเรียนตามจำนวนที่ตกลงกันได้
- การคำนวณรายได้และค่าธรรมเนียม: ระบบจะทำการคำนวณและหักค่าธรรมเนียมแพลตฟอร์ม (Platform Fee) อัตโนมัติ

7. การจัดการการประเมินผล (Review Management)

- ระบบรีวิวและให้คะแนน (Rating & Review): นักเรียนที่ตกลงจ้างงานตัวเตอรผ่านระบบสำเร็จแล้ว สามารถเข้ามาให้คะแนน (1-5 ดาว) และเขียนรีวิวได้ (จำกัดสิทธิ์ 1 งานสอนต่อ 1 รีวิว)

8. ระบบการจัดการหลังบ้านและสถิติ (Admin Operations & Analytics)

- การตรวจสอบยืนยันตัวตน, การจัดการเนื้อหาและผู้ใช้, แดชบอร์ดสรุปสถิติ

3.5 การออกแบบฐานข้อมูล (Database Schema)

รายละเอียดโครงสร้าง Entity ก่อนทำการ Normalization

User (user_id, name, email, password_hash, role, user_profile, created_at)

รหัสผู้ใช้ (user_id INT, PK, AUTO_INCREMENT), ชื่อ-นามสกุล (name VARCHAR(50), NOT NULL), อีเมลสำหรับเข้าสู่ระบบ (email VARCHAR(100), UNIQUE, NOT NULL), รหัสผ่านที่เข้ารหัสแล้ว (password_hash VARCHAR(255), NOT NULL), แบ่งสิทธิ์การใช้งาน (role ENUM('admin','student','tutor'), NOT NULL), รหัสเก็บ URL ของรูปภาพ (user_profile VARCHAR(512) NULL), วันที่เวลาที่สมัครสมาชิก (created_at DATETIME, DEFAULT NOW())

Student_Profile (student_id, user_id, grade_level, school_name)

รหัสโปรไฟล์นักเรียน (student_id INT, PK, AUTO_INCREMENT), อ้างอิงผู้ใช้งาน (user_id INT, FK → User, UNIQUE, NOT NULL), ระดับชั้นเรียน (grade_level VARCHAR(50)), ชื่อโรงเรียนหรือมหาวิทยาลัย (school_name VARCHAR(100))

Student_Post (post_id, student_id, subject, description, budget, status, created_at)

รหัสโพสต์ (post_id INT, PK, AUTO_INCREMENT), เจ้าของโพสต์ (student_id INT, FK → Student_Profile, NOT NULL), วิชาที่ต้องการเรียน (subject VARCHAR(100), NOT NULL), รายละเอียดเพิ่มเติม (description TEXT), งบประมาณที่จ่ายได้ต่อชั่วโมง (budget DECIMAL(10,2), NOT NULL, CHECK(budget > 0)), สถานะของโพสต์ (status ENUM('open','closed'), DEFAULT 'open'), วันที่ตั้งโพสต์ (created_at DATETIME, DEFAULT NOW())

Tutor_Profile (tutor_id, user_id, bio, experience, verification_status, hourly_rate)

รหัสโปรไฟล์ติวเตอร์ (tutor_id INT, PK, AUTO_INCREMENT), อ้างอิงผู้ใช้งาน (user_id INT, FK → User, UNIQUE, NOT NULL), ข้อความแนะนำตัว (bio TEXT), ประสบการณ์สอน (experience TEXT), สถานะการตรวจสอบ (verification_status ENUM('pending','verified','rejected'), DEFAULT 'pending'), เหนียวราคาที่รับสอนต่อชั่วโมง (hourly_rate DECIMAL(10,2), NOT NULL, CHECK(hourly_rate > 0))

Tutor_Subject (id, tutor_id, subject) / **Tutor_Certificate** (id, tutor_id, certificate)

Application (app_id, post_id, tutor_id, status, applied_at)

รหัสการสมัครงานสอน (app_id INT, PK, AUTO_INCREMENT), อ้างอิงโพสต์ (post_id INT, FK → Student_Post, NOT NULL), อ้างอิงติวเตอร์ (tutor_id INT, FK → Tutor_Profile, NOT NULL), สถานะ (status ENUM('pending','accepted','rejected'), DEFAULT 'pending'), วันที่เวลาที่สมัคร (applied_at DATETIME, DEFAULT NOW())

[FIX] CONSTRAINT uq_application UNIQUE (post_id, tutor_id) — ป้องกันติวเตอร์กดสมัครโพสต์เดิมซ้ำ

Tutor_Schedule (schedule_id, tutor_id, day_of_week, start_time, end_time, is_booked)

Schedule_Booking (booking_id, schedule_id, app_id)

Review (review_id, student_id, tutor_id, app_id, rating, comment, created_at)

[FIX] CONSTRAINT uq_review_per_app UNIQUE (app_id) — ป้องกันนักเรียนรีวิวติวเตอร์คนเดิมซ้ำใน 1 งาน

Payment (payment_id, student_id, tutor_id, app_id, schedule_id, amount, platform_fee, status, payment_date)

Logic: platform_fee = amount × 0.10, รายได้สุทธิที่ติวเตอร์ได้รับ (Net Earnings) = amount - platform_fee

3.5.1 Database Normalization

- **Third Normal Form (3NF)**

1. ตาราง **Review** — ตัด student_id และ tutor_id ออก เนื่องจากระบบสามารถสืบค้นได้จาก app_id อยู่แล้ว การเก็บข้อมูลผู้ใช้ซ้ำจะทำให้เกิด Transitive Dependency
2. ตาราง **Payment** — ตัด student_id, tutor_id และ schedule_id ออก เช่นเดียวกับตารางรีวิว ข้อมูลฝั่งผู้ชำระเงินและผู้รับเงินสามารถอ้างอิงได้โดยตรงจาก app_id

- **BCNF (Boyce-Codd Normal Form)**

1. ตาราง **Tutor_Subject** — ตัดคอลัมน์ id ออก เปลี่ยนมาใช้ Composite Primary Key (tutor_id, subject) การใช้ Surrogate Key ไม่สามารถป้องกันการกรอกวิชาซ้ำของติวเตอร์คนเดิมได้
2. ตาราง **Tutor_Certificate** — ตัดคอลัมน์ id ออก เปลี่ยนมาใช้ Composite Primary Key (tutor_id, certificate) ใช้หลักการเดียวกับวิชาที่สอน
3. ตาราง **Application** — เพิ่มข้อกำหนด Unique Key (UK) ที่คอลัมน์ post_id และ tutor_id เพื่อป้องกันไม่ให้ติวเตอร์กดสมัครงาน (Apply) ในโพสต์เดิมซ้ำมากกว่า 1 ครั้ง

- **Fourth Normal Form (4NF)**

1. ตาราง **Tutor_Profile** — แยกคอลัมน์ experience ออกจากตารางโปรไฟล์ แล้วสร้างเป็นตารางใหม่ชื่อ Tutor_Experience (ประกอบด้วย experience_id, tutor_id, experience_detail) เพราะประสบการณ์การสอนของติวเตอร์เป็นข้อมูลหลายค่าอิสระ (Multi-valued Dependency)

3.5.2 Database Optimization (Static)

- การออกแบบโครงสร้างสถานะบัญชีและระบบรักษาความปลอดภัย
 1. ตาราง User — เพิ่ม account_status ENUM ('active', 'suspended', 'ban'), status_updated_at, status_reason เพื่อแยกส่วนการควบคุมการเข้าถึง (Access Control) ออกจากข้อมูลโปรไฟล์ส่วนตัว
 2. Role Management — กำหนดให้จัดเก็บคอลัมน์ role (ENUM: 'student', 'tutor', 'admin') ไว้ในตารางผู้ใช้งานหลัก โดยบังคับข้อกำหนดทางธุรกิจให้ 1 บัญชีผู้ใช้ได้เพียง 1 บทบาท เพื่อลดความซับซ้อนและหลีกเลี่ยง Over-normalization
- การออกแบบระบบบันทึกประวัติการดำเนินการ (Audit Trail & Activity Logging)
 1. สร้างตาราง User_Action_Log (log_id, user_id, action_type, reason, performed_by, created_at) เพื่อบันทึกประวัติการเปลี่ยนแปลงสถานะบัญชี แยกออกจากตาราง User เพื่อรักษาขนาดของตารางให้เล็กและทำงานได้รวดเร็ว (I/O Efficiency) และรองรับ Enterprise Auditability
- การออกแบบตารางโปรไฟล์
 1. ตาราง Tutor_Profile — เพิ่ม profile_picture_url, verification_status, verification_submitted_at, verified_by, verified_at, reject_reason เพื่อรองรับ Verification Workflow ที่รัดกุม
- การออกแบบระบบยืนยันตัวตนแบบไร้สถานะ (Stateless Authentication Architecture)

เลือกใช้งานระบบ JWT โดยไม่ทำการสร้างตาราง User_Session หรือตารางจัดเก็บ Token ใดๆ เพื่อลดภาระคอขวดของฐานข้อมูล (Reduce Database I/O Bottleneck) ทำให้ Backend ตรวจสอบสิทธิ์ได้ทันที (Zero Database Hit for Authorization)

- ระบบควบคุมการแสดงผลเนื้อหา (Content Moderation & Soft Delete)

เพิ่ม is_hidden และ moderation_reason ในตาราง Post และ Review เพื่อรองรับการจัดการเนื้อหาที่ไม่เหมาะสมโดยไม่ต้องลบข้อมูลออกจากฐานข้อมูล

3.5.3 รายละเอียดโครงสร้าง Entity หลังทำการ Normalization & Database Optimization

1. **users** (user_id, name, email, password_hash, role, created_at, account_status, status_updated_at, status_reason)

ตารางกลางที่เก็บข้อมูลบัญชีหลักของผู้ใช้ทุกคนในระบบ ไม่ว่าจะเป็นนักเรียน ตัวเเตอร์ หรือแอดมิน role เป็น ENUM เก็บอยู่ในตารางเดียว ใช้หลัก Mutually Exclusive Subtype ทำให้ 1 บัญชีมีได้เพียง 1 บทบาท เป็น root ของทุก FK ในระบบ

2. **student_profiles** (student_id, user_id, grade_level, school_name, education_level)

แยกข้อมูลเฉพาะทางของนักเรียนออกจากตาราง users รองรับ Role-based Profile Specialization

3. **tutor_profiles** (tutor_id, user_id, bio, hourly_rate, verification_status, profile_picture_url, verification_submitted_at, verified_by, verified_at, reject_reason)

แยกข้อมูลเฉพาะทางของตัวเเตอร์ออกมา เนื่องจากตัวเเตอร์มี Verification Workflow ที่ซับซ้อนกว่านักเรียน profile_picture_url บังคับ NOT NULL — ตัวเเตอร์ต้องอัปโหลดรูปหน้าคนเสมอ

4. **tutor_experiences** (experience_id, tutor_id, experience_detail)

[4NF] ถูกแยกออกมาจากตารางหลักของ Tutor_Profile เพื่อรองรับหลายประสบการณ์

5. **tutor_subjects** (tutor_id, subject) — Composite PK

[BCNF] ตัด Surrogate Key id ออก เปลี่ยนมาใช้ Composite PK (tutor_id, subject) เพื่อป้องกันการบันทึกวิชาซ้ำของตัวเเตอร์คนเดิม

6. **student_posts** (post_id, student_id, subject, grade_level, learning_format, location, preferred_time, description, budget, status, created_at, is_hidden, moderation_reason)

กระดานประกาศหาติวเตอร์ learning_format (online / onsite / both),
is_hidden + moderation_reason รองรับ Soft Delete โดย Admin

7. applications (app_id, post_id, tutor_id, status, applied_at, teaching_status)

Junction Table สลายความสัมพันธ์แบบ Many-to-Many ระหว่าง
student_posts และ tutor_profiles
[BCNF] CONSTRAINT uq_applications_post_tutor UNIQUE (post_id, tutor_id) —
ป้องกันติวเตอร์กดสมัครโพสต์เดิมซ้ำ

teaching_status (not_started / ongoing / completed) ใช้เป็น Review Gate

8. tutor_schedules (schedule_id, tutor_id, day_of_week, start_time,
end_time)

ตัด is_booked ออก (ตามหลัก 3NF) เพราะเป็น Derived Data

9. schedule_bookings (booking_id, schedule_id, app_id) — UNIQUE
(schedule_id, app_id)

Conflict Prevention Layer ป้องกันไม่ให้เวลาเรียนทับซ้อนกัน

10. reviews (review_id, app_id, rating, comment, created_at, is_hidden,
moderation_reason)

[3NF] ตัด student_id และ tutor_id ออก เนื่องจากเป็น Transitive
Dependency

app_id มี UNIQUE constraint — 1 งานสอนรีวิวได้ 1 ครั้ง ป้องกันรีวิวลอมน

11. payments (payment_id, app_id, amount, platform_fee, status,
payment_date, slip_url, verified_by, verified_at, remarks)

[3NF] ตัด student_id, tutor_id และ schedule_id ออก อ้างอิงได้ผ่าน app_id
platform_fee CHECK (= ROUND(amount * 0.10, 2)) บังคับโดย DB constraint

12. wallets (wallet_id, user_id, balance, status, created_at, updated_at)

กระเป๋าเงินดิจิทัลของผู้ใช้ทุกคน user_id มี UNIQUE constraint — 1 user มีได้
1 wallet

13. transaction_logs (transaction_id, wallet_id, transaction_type, amount,
balance_after, reference_type, reference_id, description, transaction_date)

บันทึกประวัติการเงินทุกรายการแบบ immutable transaction_type ENUM
(deposit / withdrawal / payment / refund / platform_fee / tutor_earnings)

14. user_action_logs (log_id, user_id, action_type, target_type, target_id, reason, performed_by, created_at)

[Optimization] แยกออกจากตาราง User เพื่อป้องกัน Data Bloat และรองรับ Enterprise Auditability

15. reports (report_id, reporter_id, target_type, target_id, title, description, status, created_at)

ช่องทางให้ผู้แจ้งปัญหา status (pending / investigating / resolved / dismissed)

16. user_bank_accounts (bank_account_id, user_id, bank_name, account_number, account_name, is_primary)

เก็บข้อมูลบัญชีธนาคารสำหรับถอนเงินออกจาก Wallet

3.5.4 สรุป Cardinality (ความสัมพันธ์ของตาราง)

กลุ่มที่ 1 ผู้ใช้งานและข้อมูลส่วนตัว (User Management)

- users → student_profiles (1:1): 1 User สามารถมีโปรไฟล์นักเรียนได้ 1 โปรไฟล์
- users → tutor_profiles (1:1): 1 User สามารถมีโปรไฟล์ติวเตอร์ได้ 1 โปรไฟล์
- users → wallets (1:1): 1 User มีกระเป๋าสตางค์ (Wallet) ได้ 1 ใบ
- users → user_bank_accounts (1:M): 1 User สามารถผูกบัญชีธนาคารสำหรับถอนเงินได้หลายบัญชี

กลุ่มที่ 2 ข้อมูลเชิงลึกของติวเตอร์ (Tutor Details)

- tutor_profiles → tutor_experiences (1:M): 1 ติวเตอร์ สามารถเพิ่มประวัติประสบการณ์สอนได้หลายรายการ
- tutor_profiles → tutor_subjects (1:M): 1 ติวเตอร์ สามารถรับสอนได้หลายวิชา
- tutor_profiles → tutor_schedules (1:M): 1 ติวเตอร์ สามารถตั้งช่วงเวลาว่างสำหรับการสอนได้หลายช่วงเวลา

กลุ่มที่ 3 ประกาศหาติวเตอร์และการสมัคร (Posts & Applications)

- student_profiles → student_posts (1:M): 1 นักเรียน สามารถตั้งโพสต์ประกาศหาติวเตอร์ได้หลายโพสต์
- student_posts → applications (1:M): 1 โพสต์ สามารถมีติวเตอร์เข้ามาสมัครได้หลายคน
- tutor_profiles → applications (1:M): 1 ติวเตอร์ สามารถยื่นสมัครสอนในหลายๆ โพสต์ได้

(ตาราง applications ทำหน้าที่เป็น Junction Table เพื่อสลายความสัมพันธ์แบบ Many-to-Many ระหว่าง Post และ Tutor)

กลุ่มที่ 4 การเรียนการสอนและการประเมินผล (Teaching & Reviews)

- applications → schedule_bookings (1:M): 1 งานสอน ที่ตกลงกันแล้ว สามารถมีการจองคิวเวลาเรียนได้หลายช่วงเวลา
- applications → reviews (1:1): 1 งานสอน จะได้รับการรีวิวหรือให้คะแนนได้เพียง 1 ครั้งเท่านั้น

กลุ่มที่ 5 การเงิน (Finance)

- applications → payments (1:M): 1 งานสอน อาจมีการบันทึกการชำระเงินได้หลายรายการ
- wallets → transaction_logs (1:M): 1 กระเป๋าเงิน มีประวัติการทำธุรกรรมได้หลายรายการ

กลุ่มที่ 6 ระบบความปลอดภัยและการตรวจสอบ (System & Audit)

- users → reports (1:M): 1 User สามารถถกตรีพอร์ตปัญหาได้หลายครั้ง
- users → user_action_logs (1:M): 1 User (ที่เป็น Admin) สามารถดำเนินการจัดการระบบและสร้าง Log การกระทำได้หลายรายการ

3.6 System Integrity (Dynamic)

3.6.1 ระบบป้องกันการรีวิวซ้ำ (Review Gating Logic)

Backend จะทำหน้าที่ตรวจสอบสถานะ `teaching_status` จากตาราง Application ก่อนอนุญาตให้คำสั่ง `INSERT` ลงตาราง Review ทำงาน เพื่อป้องกันการรีวิวปลอมหรือการรีวิวก่อนจบงาน

3.6.2 ระบบป้องกันการโจมตีและจำกัดโควตา (Rate Limiting & Anti-Bot)

ทุกครั้งที่มีการโพสต์หาติวเตอร์ Backend จะทำการนับจำนวน Record ในตาราง Post ของผู้ใช้นั้นในรอบวัน หากเกิน 10 ครั้ง Backend จะสั่งอัปเดตสถานะ `account_status` ในตาราง User เป็น `suspended` ทันที

3.6.3 ระบบรักษาสถานะและความปลอดภัย (Admin Moderation Logic)

เมื่อ Admin ทำการระงับการใช้งานหรือซ่อนเนื้อหา Backend จะทำหน้าที่ประสานงาน (Atomic Transaction) ระหว่างการเปลี่ยนสถานะ `is_hidden` ในตาราง เป้าหมาย พร้อมกับการบันทึกประวัติลงในตาราง `Admin_Action_Log` พร้อมกัน

บทที่ 4 การพัฒนาและผลการทำงานของระบบ (Implementation & Results)

4.1 ตัวอย่าง Use Case หลักของระบบ

Use Case 1: โปสต์หาติวเตอร์

- ผู้ใช้งานหลัก: นักเรียน
- เงื่อนไขก่อนเริ่ม: ผู้ใช้ทำการเข้าสู่ระบบเรียบร้อยแล้ว
- เหตุการณ์หลัก: นักเรียนกรอกข้อมูลวิชาที่ต้องการเรียน ระบุรายละเอียดเพิ่มเติม และกำหนดงบประมาณ จากนั้นระบบจะทำการบันทึกข้อมูลและแสดงโปสต์ให้ติวเตอร์เห็น
- ผลลัพธ์: ระบบบันทึกโปสต์สำเร็จและแสดงข้อมูลในหน้าจกระดานรายการงาน

Use Case 2: สมัครงานสอน

- ผู้ใช้งานหลัก: ติวเตอร์
- เงื่อนไขก่อนเริ่ม: ติวเตอร์ทำการเข้าสู่ระบบและมีโปรไฟล์ในระบบเรียบร้อยแล้ว
- เหตุการณ์หลัก: ติวเตอร์ค้นหางานสอนที่สนใจ เมื่อพบงานที่ต้องการแล้วจึงกดสมัครงาน จากนั้นระบบจะบันทึกการสมัคร
- ผลลัพธ์: ระบบบันทึกข้อมูลการสมัครงานสำเร็จ และทำการแจ้งเตือนไปยังนักเรียนเจ้าของโปสต์

4.2 System Flow

Flow 1: การเริ่มต้นใช้งาน (Onboarding Flow)

เป้าหมาย: ผู้ใช้เข้ามาในระบบและพร้อมใช้งาน

- **สมัครสมาชิก (Register):** ผู้ใช้ใหม่เข้ามาที่หน้าเว็บ เลือก role (Student หรือ Tutor) กรอกชื่อ อีเมล รหัสผ่าน ระบบจะสร้าง Account และ Profile เปล่าๆ เตรียมไว้ให้
 - Tutor: เมื่อสมัครเสร็จสิ้น ระบบจะกำหนดสถานะการยืนยันตัวตนไว้ที่ค่า `verification_status = 'pending'` โดยอัตโนมัติ เพื่อรอให้ Admin ตรวจสอบ
- **เข้าสู่ระบบ (Login):** ผู้ใช้กรอกอีเมลและรหัสผ่านเพื่อรับสิทธิ์การเข้าถึง (Token/Session) ระบบจะพาไปยังหน้า Dashboard ตามบทบาทของตัวเอง

- **จัดการโปรไฟล์ (Profile Setup):**

- นักเรียน: เข้าไปอัปเดตข้อมูลทั่วไป
- ตัวเตอร: เข้าไปอัปเดตข้อมูลและอัปโหลดรูปภาพ หากผ่านไป 1 สัปดาห์ยังไม่ทำการอัปโหลดรูปภาพ สถานะจะกลายเป็น rejected และหากผ่านไป 7 วันแล้วยังไม่มีการอัปโหลด ระบบจะทำการลบ Account นั้นออก หากตัวเตอรอัปโหลดรูปภาพ และผ่านการตรวจสอบจาก Admin แล้ว สถานะของตัวเตอรจะกลายเป็น verified

Flow 2: ระบบการจับคู่ (Core Matching Flow)

เป้าหมาย: นักเรียนและตัวเตอรตกลงเรียนกันสำเร็จ

- **ประกาศหาตัวเตอร (Post a Job):** นักเรียนสร้างโพสต์ประกาศ โดยระบุวิชาที่อยากเรียน, ระดับชั้น/ความยาก, งบประมาณ, รูปแบบการเรียน (Online/Onsite/Both), และช่วงเวลา/สถานที่ที่สะดวก ระบบจะบันทึกโพสต์และตั้งสถานะเป็น `post.status = 'open'`
- **ค้นหางาน (Browse Jobs):** ตัวเตอรที่ผ่านการยืนยันแล้ว (verified) เข้ามาที่หน้า "กระดานงาน" เพื่อดูโพสต์ของนักเรียนทั้งหมดที่มีสถานะเป็น open
- **ส่งใบสมัคร (Apply):** ตัวเตอรกดปุ่ม "สมัครสอน" ระบบจะสร้างใบสมัครและกำหนดสถานะเริ่มต้นเป็น `application.status = 'pending'`
- **พิจารณาผู้สมัคร (Review Applicants):**** นักเรียนเข้ามาดูโพสต์ของตัวเอง ระบบจะแสดงรายชื่อและโปรไฟล์ของตัวเตอรทุกคนที่กดสมัครเข้ามา
- **ตัดสินใจ (Decision):**
 - ถ้านักเรียนกด "ปฏิเสธ (Reject)" → ใบสมัครของตัวเตอรคนนั้นจะเปลี่ยนเป็น `application.status = 'rejected'`
 - ถ้านักเรียนกด "ยอมรับ (Accept)" → `application.status = 'accepted'`, `teaching_status = 'not_started'`, `post.status = 'closed'` และปรับใบสมัครของตัวเตอรคนอื่นๆ ที่เหลือในโพสต์นั้นให้เป็น rejected ทั้งหมดอัตโนมัติ

Flow 3: การจัดการเวลาและการเงิน (Scheduling & Payment Flow)

เป้าหมาย: จัดการเรื่องการเงินผ่านระบบ Escrow และสถานะการสอน

- **นักเรียนเตรียมเงิน:** นักเรียนเติมเงินเข้า Wallet (deposit)
- **นักเรียนชำระเงิน (Escrow):** เมื่อสถานะการสอนยังไม่เริ่ม (`teaching_status = 'not_started'`) นักเรียนกด "ชำระเงิน" ระบบจะหักเงินจาก Wallet ของนักเรียนและนำไปพักไว้ในระบบ (Escrow) โดยกำหนดสถานะเป็น `payments.status = 'pending'`

- **กรณียกเลิกคลาส (Cancellation):** สามารถทำได้เฉพาะตอนที่ `teaching_status = 'not_started'` และชำระเงินแล้วเท่านั้น นักเรียนกด 'ยกเลิก' ระบบจะคืนเงิน 97% กลับสู่ Wallet ของนักเรียน (หักค่าธรรมเนียม Gateway 3%) โดย Tutor จะไม่ได้รับเงิน
- **เริ่มสอน (Start Class):** ตัวเตอรืกด "เริ่มสอน" → สถานะเปลี่ยนเป็น `teaching_status = 'ongoing'`
- **สอนเสร็จสิ้น (End Class):** ตัวเตอรืกด "สอนเสร็จ" → สถานะเปลี่ยนเป็น `teaching_status = 'completed'`
- **ยืนยันและปล่อยเงิน (Release Funds):** นักเรียนกด "ยืนยันการเรียน" ระบบปล่อยเงินจาก Escrow โดยเปลี่ยนสถานะเป็น `payments.status = 'completed'` แบ่งยอดเงิน 90% โอนเข้า Wallet ของ Tutor และ 10% หักเป็นค่าบริการโอนเข้า Wallet ของ Admin (Platform)

Flow 4: การประเมินผลหลังเรียน (Review Flow)

เป้าหมาย: รักษามาตรฐานและคุณภาพของตัวเตอรื

- **พร้อมให้คะแนน:** เมื่อถึงกำหนดเวลาเรียนและสถานะเปลี่ยนเป็น `teaching_status = 'completed'`
- **ให้คะแนน (Rating & Review):** นักเรียนสามารถเข้าไปกดให้คะแนนดาว (1-5) และเขียนคอมเมนต์รีวิวการสอนของตัวเตอรืคนนั้นได้ เงื่อนไข: สามารถรีวิวได้ 1 ครั้ง ต่อ 1 Application
- **แสดงผล (Display Stats):** คะแนนเฉลี่ยจะถูกนำไปคำนวณและแสดงที่หน้าโปรไฟล์ของตัวเตอรื

Flow 5: การจัดการหลังบ้าน (Admin Operations Flow)

เป้าหมาย: ควบคุมความเรียบร้อยของแพลตฟอร์ม

- **ดูสถิติ (Dashboard):** แอดมินเข้าสู่ระบบเพื่อดูยอดผู้ใช้งานรวม, ยอดโพสต์หาตัวเตอรื, และจำนวนการจับคู่สำเร็จ
- **ตรวจสอบและอนุมัติ (Verification):** แอดมินตรวจสอบข้อมูลและรูปภาพของตัวเตอรืที่เพิ่งสมัครเข้ามา เพื่อปรับสถานะจาก `pending` เป็น `verified` หรือ `rejected`
- **จัดการผู้ใช้ (User Management):** หากพบผู้ใช้ที่ทำผิดกฎ แอดมินสามารถกดปุ่ม "Ban" เพื่อระงับบัญชีผู้ใช้นั้นได้

4.3 การทำงานของระบบหน้าบ้าน (Frontend UI)

ระบบพัฒนา Dashboard สำหรับผู้ใช้งานแต่ละบทบาท โดยดึงข้อมูลสถิติมาแสดงผลแบบเรียลไทม์ ผ่าน RESTful API ได้แก่

- **Admin Dashboard** แสดงสถิติภาพรวม: จำนวนผู้ใช้, โพสต์, ตัวเตอรืรอ verify, บัญชีถูก ban, รายงานรอดำเนินการ พร้อมฟังก์ชัน verify/reject tutor, ban user, จัดการรายงาน
- **Tutor Dashboard** แสดง: งานที่เหมาะสมกับฉัน, คลาสที่กำลังสอน, รายได้เดือนนี้ (ดึงจาก transaction_logs ที่ transaction_type = 'tutor_earnings'), คะแนนรีวิวเฉลี่ย
- **Student Dashboard** แสดงโพสต์, รายชื่อผู้สมัคร, คอร์สที่เรียนอยู่, ระบบ Wallet

4.4 การจัดการตรรกะระบบหลังบ้าน (Backend Logic)

- **Middleware การตรวจสอบสิทธิ์** ระบบใช้ @token_required decorator ตรวจสอบ JWT Token ทุก request และ @role_required decorator ตรวจสอบบทบาทของผู้ใช้ ก่อนเข้าถึง endpoint
- **การสร้าง Profile อัตโนมัติ** เมื่อสมัครสมาชิก ระบบจะสร้าง student_profile หรือ tutor_profile และ wallet ให้อัตโนมัติผ่าน Atomic Transaction
- **Payment Escrow Logic** ระบบหักเงินจาก wallet นักเรียนและพักไว้ใน payment record (status = pending) จนกว่านักเรียนจะยืนยัน จิงโอน 90% เข้า wallet ตัวเตอรื และ 10% เข้า wallet admin

4.5 การจัดการข้อผิดพลาด (Error Handling)

ระบบใช้ Database Constraints ในการป้องกันข้อผิดพลาด ตัวอย่างการจัดการ Error:

- **Error 1062 Duplicate Entry** — เกิดเมื่อตัวเตอรืพยายามสมัครโพสต์เดิมซ้ำ ระบบจะ return JSON error message "คุณได้สมัครโพสต์นี้ไปแล้ว"
- **Error 3819 Check Constraint** — เกิดเมื่อพยายามบันทึก platform_fee ที่ไม่ตรงกับสูตร $\text{ROUND}(\text{amount} * 0.10, 2)$
- **Race Condition Prevention** — ใช้ SELECT ... FOR UPDATE ล็อคแถว wallet ก่อนอัปเดตยอดเงิน ป้องกัน Dirty Read

บทที่ 5 สรุปผลและข้อเสนอแนะ (Conclusion & Future Work)

5.1 สรุปผลการดำเนินงาน

ระบบ Tutor Match สามารถบรรลุเป้าหมายหลักที่ตั้งไว้ได้ดังนี้

1. **ระบบ Matchmaking** นักเรียนและติวเตอร์สามารถจับคู่กันได้จริงผ่าน Core Matching Flow ครบทั้ง 5 ขั้นตอน ตั้งแต่การโพสต์หาติวเตอร์ไปจนถึงการรีวิวหลังเรียนจบ
2. **ความปลอดภัยสูง** ระบบมีการ Hash Password ด้วย Bcrypt, ใช้ JWT สำหรับ Stateless Authentication, มี Role-Based Access Control ทุก endpoint และมี Database Constraints ป้องกันข้อมูลขยะในระดับฐานข้อมูล
3. **ระบบการเงิน Escrow** รองรับ Payment Flow ครบถ้วน: ชำระเงิน → Escrow → เริ่มสอน → จบสอน → ยืนยัน → Payout 90/10 พร้อมระบบยกเลิกและคืนเงิน 97%
4. **การจัดการหลังบ้านมีประสิทธิภาพ** Admin Dashboard ดึงข้อมูลสถิติแบบ real-time สามารถ verify/reject tutor, ban user, จัดการรายงาน และ Soft Delete เนื้อหาที่ไม่เหมาะสมได้
5. **ฐานข้อมูลผ่าน Normalization** โครงสร้าง 16 ตารางผ่านการทำ 3NF, BCNF และ 4NF ลดความซ้ำซ้อนและป้องกัน Anomaly ครบถ้วน

5.2 ข้อจำกัดของระบบ

1. **ระบบ Payment** ขณะนี้ยังเป็นการจำลองการชำระเงินผ่าน Wallet ภายในระบบ ยังไม่ได้ต่อ API Payment Gateway ของธนาคารจริง เช่น PromptPay หรือ QR Code
2. **ระบบอัปโหลดไฟล์สลิป** ยังจำลองการ verify สลิปอยู่ ยังไม่มี OCR หรือ Automated Verification
3. **การแจ้งเตือน (Notification)** ยังไม่มีระบบแจ้งเตือน Real-time เช่น Email หรือ Push Notification เมื่อมีการสมัครงาน ยอมรับ หรือปฏิเสธ
4. **ระบบลบบัญชีอัตโนมัติ** กรณีติวเตอร์ไม่อัปโหลดรูปภาพภายใน 7 วัน ยังไม่ได้ implement ด้วย Scheduled Job จริง

5.3 ข้อเสนอแนะเพื่อการพัฒนาต่อ

1. **ระบบแชทบแบบ Real-time** เพิ่ม WebSocket หรือ Socket.IO เพื่อให้นักเรียนและติวเตอร์สื่อสารกันได้โดยตรงภายในแพลตฟอร์ม
2. **ระบบวิดีโอคอล** รวม Video Call สำหรับการเรียน Online ไว้ในตัวแพลตฟอร์ม เช่น WebRTC หรือ Third-party SDK
3. **การต่อ Payment Gateway จริง** เชื่อมกับ API ของ KBank, SCB หรือ PromptPay เพื่อรองรับการโอนเงินจริง
4. **Mobile Application** พัฒนา Mobile App สำหรับ iOS และ Android เพื่อเพิ่มความสะดวกในการใช้งาน
5. **ระบบ Automated Verification** ใช้ Machine Learning หรือ OCR ช่วยตรวจสอบเอกสารและสลิปโอนเงินอัตโนมัติ
6. **Analytics Dashboard** เพิ่มกราฟและ Data Visualization สำหรับ Admin ในการวิเคราะห์แนวโน้มตลาด วิชาที่มีความต้องการสูง และรายได้ของแพลตฟอร์ม
7. **Refund System** ยังไม่ Implement (สาเหตุที่ต้องมีการจับเวลาและ Escrow) ทางผู้พัฒนาได้ ออกแบบไว้ว่า ถ้าหากนักเรียนคนนั้นได้ทำการเรียนไปเป็นเวลา x นาที แล้วมีการยกเลิกการเรียน เงินที่นักเรียนคนนั้นกดจ่ายไปจะถูกนำมาหักตามเรทค่าแรง ต่อชั่วโมงของติวเตอร์ (คิดเทียบอัตราส่วน) แล้วทำการคืนเงินที่เหลือให้กับนักเรียนคนนั้น จากนั้นก็จะทำการโอนเงินค่าแรงที่ได้ให้กับติวเตอร์ (กันปิด) โดยระบบจะทำการเก็บค่าธรรมเนียม 7% (gate ways) จากทางฝั่งของ นักเรียน (เก็บจากเงินต้น) และเก็บค่าธรรมเนียมตามอัตราส่วนของ เวลาที่ติวเตอร์คนนั้นทำงานไป (100% เก็บ 10, 50% เก็บ 5)
8. **student_profiles** ยังไม่ทำการ Implement เข้าสู่ระบบจริงเนื่องจากทางผู้พัฒนามองว่าไม่จำเป็น student ไม่มีความจำเป็นที่จะต้องสร้างช่องโปรไฟล์ ทางผู้พัฒนาจึงใช้ mock data ในการแสดงผลเท่านั้น

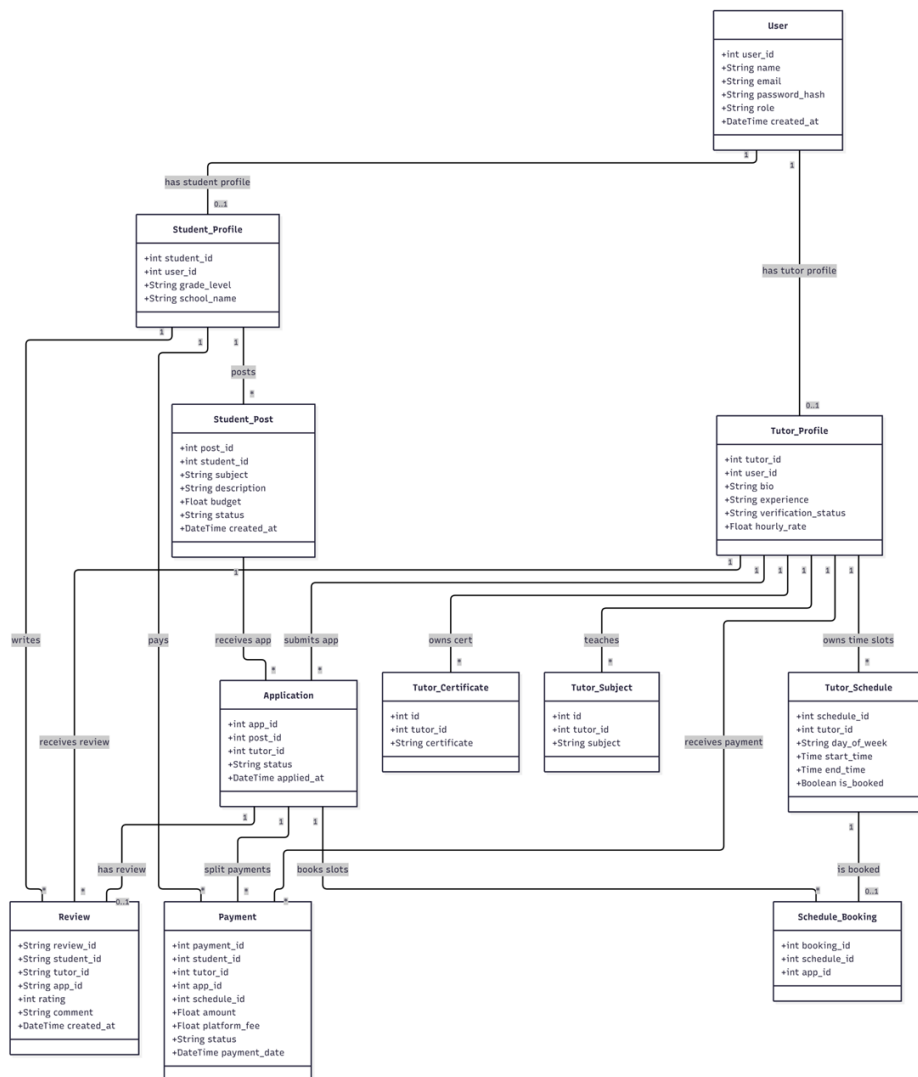
บรรณานุกรม

1. Auth0. (2024). Introduction to JSON web tokens. <https://jwt.io/introduction>
2. Date, C. J. (2003). An introduction to database systems (8th ed.). Addison-Wesley.
3. Flask Documentation. (2024). Flask - A lightweight WSGI web application framework. <https://flask.palletsprojects.com/>
4. MySQL Documentation. (2024). MySQL 8.0 reference manual. <https://dev.mysql.com/doc/refman/8.0/en/>
5. OWASP. (2024). Password storage cheat sheet. https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
6. Ramakrishnan, R., & Gehrke, J. (2002). Database management systems (3rd ed.). McGraw-Hill.

ภาคผนวก

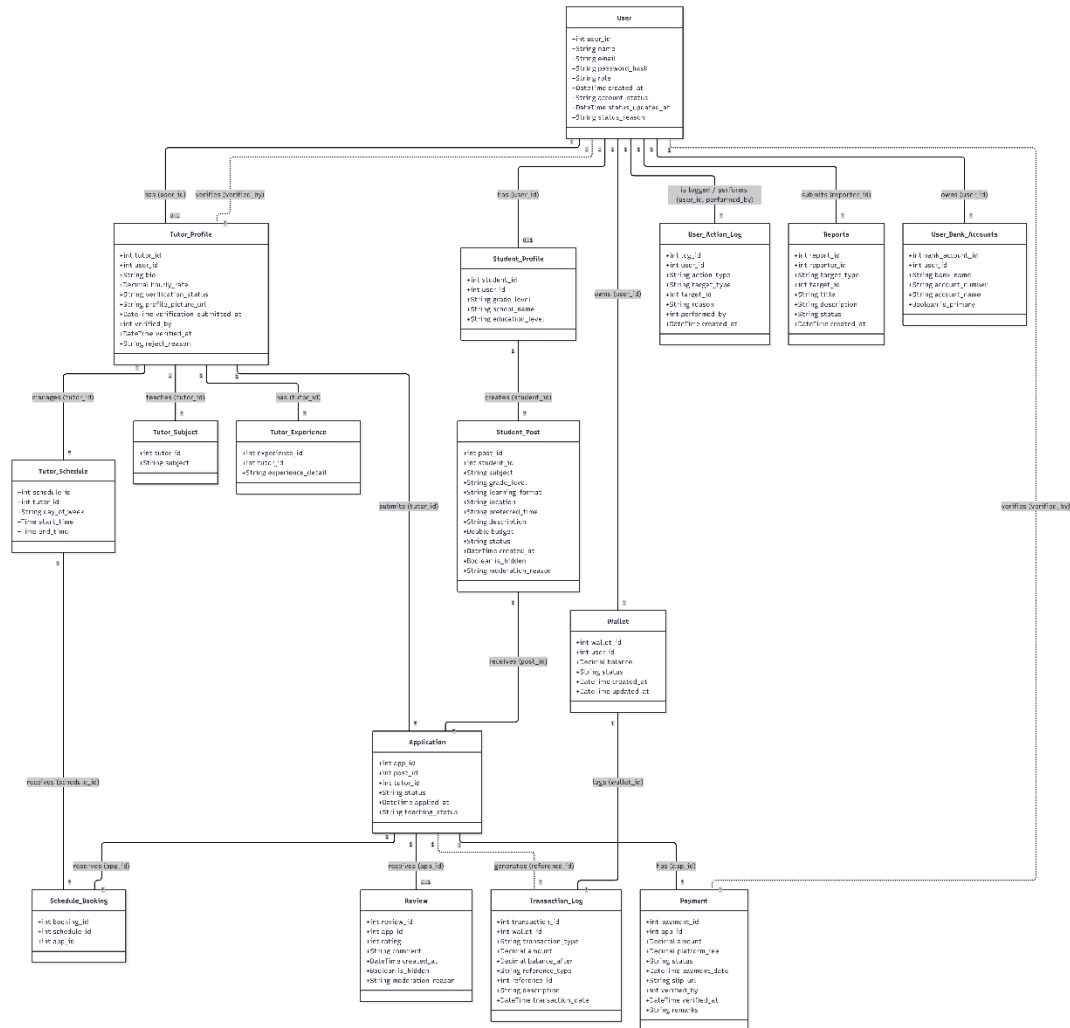
ภาคผนวก ก: E/R Diagram ก่อนทำการ Normalization

Entity หลักก่อน Normalization ได้แก่: User, Student_Profile, Student_Post, Tutor_Profile, Tutor_Subject, Tutor_Certificate, Application, Tutor_Schedule, Schedule_Booking, Review, Paymen



ภาคผนวก ข: E/R Diagram หลังทำการ Normalization

Entity หลังผ่าน Normalization และ Optimization (16 ตาราง): users, student_profiles, tutor_profiles, tutor_experiences, tutor_subjects, student_posts, applications, tutor_schedules, schedule_bookings, reviews, payments, wallets, transaction_logs, user_action_logs, reports, user_bank_accounts



ภาคผนวก ค: รายการอ้างอิงโครงสร้างตาราง (Database Schema)

โครงสร้างฐานข้อมูลทั้งหมดสามารถดูได้ที่ไฟล์ `database/schema.sql` ใน *repo*

รหัส	ชื่อตาราง	ไฟล์	บรรทัด	คำอธิบาย
SCH-01	users	schema.sql	14–22	ตารางบัญชีผู้ใช้หลัก รองรับ 3 บทบาท (admin/student/tutor) พร้อม account lifecycle
SCH-02	student_profiles	schema.sql	59–68	โปรไฟล์เฉพาะทางของนักเรียน แยกออกจาก users เพื่อรองรับ Role-based Profile Specialization
SCH-03	tutor_profiles	schema.sql	23–43	โปรไฟล์ติวเตอร์พร้อม Verification Workflow (pending/verified/rejected)
SCH-04	tutor_experience	schema.sql	45–53	[4NF] แยกจาก tutor_profiles เพื่อรองรับหลายประสบการณ์ต่อ 1 ติวเตอร์
SCH-05	tutor_subjects	schema.sql	55–63	[BCNF] Composite PK (tutor_id, subject) ป้องกันการบันทึกวิชาซ้ำของติวเตอร์คนเดิม
SCH-06	student_posts	schema.sql	70–84	กระดานประกาศหาติวเตอร์ รองรับ Soft Delete ผ่าน is_hidden + moderation_reason
SCH-07	applications	schema.sql	86–100	[BCNF] Junction Table ระหว่าง student_posts และ tutor_profiles มี UNIQUE (post_id, tutor_id)
SCH-08	tutor_schedules	schema.sql	102–112	[3NF] ตัด is_booked ออกเพราะเป็น Derived Data สามารถคำนวณได้จาก schedule_bookings

รหัส	ชื่อตาราง	ไฟล์	บรรทัด	คำอธิบาย
SCH-09	schedule_bookings	schema.sql	114–125	Conflict Prevention Layer ป้องกันการจองเวลาเรียนซ้ำซ้อน มี UNIQUE (schedule_id, app_id)
SCH-10	reviews	schema.sql	127–139	[3NF] ตัด student_id/tutor_id ออก อ้างอิงผ่าน app_id เท่านั้น, UNIQUE (app_id) ป้องกันรีวิวซ้ำ
SCH-11	payments	schema.sql	141–159	[3NF] ตัด student_id/tutor_id ออก, CHECK constraint บังคับ platform_fee = ROUND(amount*0.10,2)
SCH-12	wallets	schema.sql	161–170	กระเป๋าเงินดิจิทัล UNIQUE (user_id) บังคับ 1 user ต่อ 1 wallet
SCH-13	transaction_logs	schema.sql	172–184	บันทึกประวัติการเงินแบบ immutable รองรับ 6 ประเภท (deposit/withdrawal/payment/refund/platform_fee/tutor_earnings)
SCH-14	user_action_logs	schema.sql	186–198	[Optimization] Audit Trail แยกออกจาก users ป้องกัน Data Bloat รองรับ Enterprise Auditability
SCH-15	reports	schema.sql	200–212	ช่องทางแจ้งปัญหา รองรับ 4 สถานะ (pending/investigating/resolved/dismissed)
SCH-16	user_bank_accounts	schema.sql	214–222	บัญชีธนาคารสำหรับถอนเงินออกจาก Wallet รองรับ 1 user มีได้หลายบัญชี

ภาคผนวก ง: รายการอ้างอิง SQL Queries

SQL Query ที่ใช้ในระบบ ฉบับเต็ม สามารถดูได้ที่ไฟล์ database/complex_queries.sql ใน repo

รหัส	Level	ชื่อ Query	ตารางที่ใช้	บรรทัด	คำอธิบาย
Q-01	1	COUNT ผู้ใช้ทั้งหมด	users	18–21	นับจำนวนผู้ใช้งานทั้งหมดในระบบ ไม่มีเงื่อนไข
Q-02	1	ค้นหา User จากอีเมล	users	26–30	ค้นหาข้อมูล user จาก email ใช้ตอน login ผ่าน index บน email
Q-03	1	ดึง student_id จาก user_id	student_profiles	35–40	แปลง user_id → student_id ก่อนดำเนินการในส่วนที่ต้องการ student_id
Q-04	1	ดึง Wallet พร้อม Row Lock	wallets	45–51	SELECT ... FOR UPDATE ล็อคแถวก่อนอัปเดตยอดเงิน ป้องกัน Race Condition
Q-05	1	ดึงประวัติ Transaction	transaction_logs	56–70	ดึงรายการธุรกรรมของ wallet เรียงใหม่สุดก่อน

รหัส	Level	ชื่อ Query	ตารางที่ใช้	บรรทัด	คำอธิบาย
Q-06	1	อัปเดตสถานะบัญชี	users	74-80	พร้อม DATE_FORMAT เปลี่ยน account_status เป็น ban/suspended/active ผ่าน parameterized query
Q-07	1	อัปเดตสถานะการสอน	applications	84-90	เปลี่ยน teaching_status (not_started → ongoing → completed) ทีละขั้น
Q-08	1	ลบรีวิว (Hard Delete)	reviews	94-99	ลบรีวิวตรงๆ โดยตรวจสอบสิทธิ์ใน Python ก่อนรัน query
Q-09	1	ดึง tutor_id จาก user_id	tutor_profiles	241-247	แปลง user_id → tutor_id คล้าย Q03 แต่ ใช้กับตาราง tutor_profiles เพื่อให้ระบบระบุ

รหัส	Level	ชื่อ Query	ตารางที่ใช้	บรรทัด	คำอธิบาย
Q-10	1	อนุมัติ/ปฏิเสธ Tutor	tutor_profiles	251–261	ตัวติวเตอร์ก่อนดำเนินการ UPDATE tutor_profiles ตั้ง verification_status, verified_by, verified_at, reject_reason อ้างอิงด้วย user_id ของติวเตอร์
Q-11	2	โปรไฟล์ติวเตอร์ + ข้อมูลผู้ใช้	tutor_profiles, users	111–125	JOIN tutor_profiles และ users เพื่อรวมข้อมูลโปรไฟล์การสอนกับข้อมูลบัญชี
Q-12	2	รายชื่อผู้ใช้ทั้งหมด (Admin)	users, tutor_profiles	129–144	LEFT JOIN เพราะ student ไม่มีแถวใน tutor_profiles, แสดง NULL สำหรับคอลัมน์ tutor
Q-13	2	รายการรายงาน + ชื่อผู้แจ้ง	reports, users	148–164	JOIN reports → users เพื่อแสดงชื่อผู้รายงาน

รหัส	Level	ชื่อ Query	ตารางที่ใช้	บรรทัด	คำอธิบาย
Q-14	2	โพสต์เปิดรับสมัคร + ชื่อนักเรียน	student_posts, student_profiles, users	168– 188	รายงานแทน reporter_id ดิบ JOIN 3 ตาราง กรอง status='open' และ is_hidden=FALSE เพื่อแสดงกระดา นงาน
Q-15	2	ประวัติใบสมัครของตัวเตอร์	applications, student_posts, student_profiles, users	192– 213	Chain JOIN 4 ตารางเพื่อดูว่าแ ตละใบสมัครเป็น โพสต์ของนักเรีย นคนไหน
Q-16	2	รายชื่อ Tutor รอ Verification	tutor_profiles, users	217– 237	JOIN tutor_profiles + users กรอง verification_st atus='pending' ORDER BY verification_su bmitted_at ASC เพื่ออนุมัติ ตามลำดับเวลา
Q-17	2	ตารางสอน (accepted) + สถานะชำระเงิน	applications, student_posts, student_profiles, users, payments	265– 289	LEFT JOIN payments เพราะบางงานยัง ไม่ได้ชำระ

รหัส	Level	ชื่อ Query	ตารางที่ใช้	บรรทัด	คำอธิบาย
					หากใช้ INNER JOIN งานที่ยังไม่จ่ายจะหาย
Q-18	3	สรุป Rating แยกตามดาว	reviews, applications, tutor_profiles	301–318	SUM(CASE WHEN rating=N THEN 1 ELSE 0) ทำ Conditional COUNT แยกแต่ละดาวพร้อม AVG รวม
Q-19	3	รายการสมัครพร้อมสถานะชำระ (ภาษาไทย)	applications, student_posts, student_profiles, tutor_profiles, users, payments	322–348	CASE WHEN แปลง payment status → ข้อความไทย, JOIN users 2 ครั้งด้วย alias u_student/u_tutor
Q-20	3	ติวเตอร์ Rating >= 4 ดาว	tutor_profiles, users, applications, reviews	352–370	GROUP BY รวมรีวิวต่อติวเตอร์, HAVING กรองหลัง aggregate (ต่างจาก WHERE ที่กรองก่อน)

รหัส	Level	ชื่อ Query	ตารางที่ใช้	บรรทัด	คำอธิบาย
Q-21	3	ติวเตอร์มีใบสมัคร Pending > 1 งาน	tutor_profiles, users, applications	374– 388	COUNT ใบสมัคร GROUP BY ติวเตอร์, HAVING COUNT > 1 กรองเฉพาะคนที่ มีงานค้างหลาย งาน
Q-22	3	รายได้รวมแยกตามติวเตอร์	payments, applications, tutor_profiles, users	392– 409	SUM amount, platform_fee, net_earnings คำนวณใน query กรองเฉพาะ payment ที่ completed
Q-23	4	Wallet ทุก User (COALESCE)	users, wallets	421– 435	LEFT JOIN + COALESCE(bal ance, 0.00) แปลง NULL → 0 สำหรับ user ที่ยังไม่มี wallet
Q-24	4	นักเรียนที่ตั้งงบประมาณสูงกว่าค่าเฉลี่ย	student_posts, student_profiles, users	439– 455	Subquery ใน WHERE: WHERE budget > (SELECT AVG(budget))

รหัส	Level	ชื่อ Query	ตารางที่ใช้	บรรทัด	คำอธิบาย
					FROM student_posts) รันครั้งเดียว
Q-25	4	สรุปยอดรับ-จ่ายต่อ User	users, wallets, transaction_logs	459– 475	CASE WHEN แยก transaction เป็น total_in/total_ out แล้ว SUM แต่ละฝั่ง
Q-26	4	นักเรียนที่เคยเขียนรีวิว	users, student_profiles, student_posts, applications, reviews	479– 497	Subquery แบบ IN ติดตาม FK chain 4 ตาราง: student_profil es → posts → applications → reviews
Q-27	4	รีวิวทั้งหมด (5 JOINS)	reviews, applications, tutor_profiles, student_posts, student_profiles, users	501– 524	JOIN 5 ตาราง ใช้ users 2 ครั้ง (u=นักเรียน, tu=ติวเตอร์) เพื่อดึงชื่อทั้งสอง ฝ่าย
Q-28	5	Admin Dashboard (7 ตัวชี้วัด)	users, student_posts, tutor_profiles, reports	536– 548	Scalar Subquery 7 ตัวใน SELECT เดียว ได้ผลลัพธ์ 1 row

รหัส	Level	ชื่อ Query	ตารางที่ใช้	บรรทัด	คำอธิบาย
					แผนการรัน query แยก 7 ครั้ง
Q-29	5	Tutor Dashboard (4 ตัวชี้วัด)	applications, student_posts, reviews	552–574	Correlated Subquery แต่ละตัวรับ tutor_id จาก outer, monthly_income ใช้ MONTH()+YEAR()
Q-30	5	ติวเตอร์ Verified + avg_rating	tutor_profiles, users, applications, reviews	578–599	LEFT JOIN reviews พร้อม AND r.is_hidden=FALSE ใน JOIN condition ให้ติวเตอร์ไม่มีรีวิวยังอยู่ในผล
Q-31	5	คอร์สทั้งหมดของนักเรียน (query ที่ซับซ้อนที่สุด)	applications, student_posts, tutor_profiles, users, payments, reviews	603–672	JOIN 6+2 ตาราง + Correlated Scalar Subquery 4 ตัว: GROUP_CONCAT subjects, experiences,

รหัส	Level	ชื่อ Query	ตารางที่ใช้	บรรทัด	คำอธิบาย
Q-32	5	ยกเลิกการจอง (Subquery ใน UPDATE)	student_posts, applications	676-688	avg_rating, review_count UPDATE student_posts โดยใช้ Subquery ใน WHERE เพื่อหา post_id จาก app_id
Q-33	5	Audit Log ประวัติการกระทำ Admin	user_action_logs, users	698-710	JOIN users 2 ครั้ง (u_target=เป้า หมาย, u_admin=ผู้ดำ เนินการ), LEFT JOIN สำหรับ target ที่ไม่ใช่ user